

DATE: 24-03-2026

TOPIC: SEMAPHORES (PRODUCER & CONSUMER)

OUTPUT:

A terminal window titled 'me@localhost: ~/sphore' showing the execution of a semaphore-based producer-consumer program. The terminal displays a directory listing of files like 'bankers', 'mai', 'music', 'priority.c', 'sh', and 'wrtllw.sh'. It then shows the user creating a directory 'sphore', changing to it, compiling 'bestfit.c' with 'gcc', and running './bestfit'. The output shows 4 items produced and 4 consumed. Finally, the user opens 'bestfit.c' with 'vi'.

```
me@localhost: ~/sphore
- bankers mai music priority.c sh wrtllw.sh
a bin mail.c neyk.c Public xjf xvz
ABC close meyk os_lab1 REC sjf.c
abc.sh Desktop moithu os_lab1.c roundrobin Templates
ab.sh Documents mai os_lab.c roundrobin.c test.sh
bank Downloads dev Picturas sample.c Videos
hanker.c h mey priority sample.tet while.sh

[me@localhost sphore ~]$ mkdir sphore
[me@localhost sphore ~]$ cd sphore
[me@localhost sphore]$
[me@localhost sphore]$ gcc bestfit.c -o bestfit
[me@localhost sphore]$ ./bestfit
Produced: 0
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Consumed: 0
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
[me@localhost sphore]$ vi bestfit.c
[me@localhost sphore]$
```

```
Activities Terminal Mar 24 11:03
mey@localhost:~/bestfit — /usr/bin/vim bestfit.c

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
sem_t mutex;

void *producer(void *arg)
{
    int item;
    for(int i = 0; i < BUFFER_SIZE; i++)
    {
        item = i;
        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&full);
    }
}

void *consumer(void *arg)
```

```
Activities Terminal Mar 24 22:04
mey@localhost:~/bestfit — /usr/bin/vim bestfit.c

    {
        int item;

        for(int i = 0; i < BUFFER_SIZE; i++)
        {
            sem_wait(&full);
            sem_wait(&mutex);

            item = buffer[out];
            printf("Consumed: %d\n", item);
            out = (out + 1) % BUFFER_SIZE;

            sem_post(&mutex);
            sem_post(&empty);
        }
    }

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
}
```